



University of Antwerp
| Faculty of Science

Reinforcement Learning

Markov Decision Processes

Marco Favier

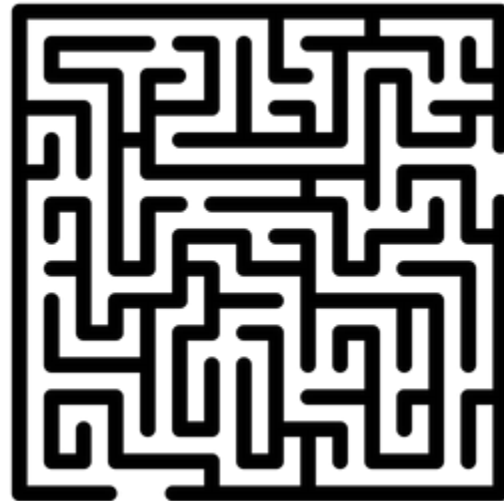
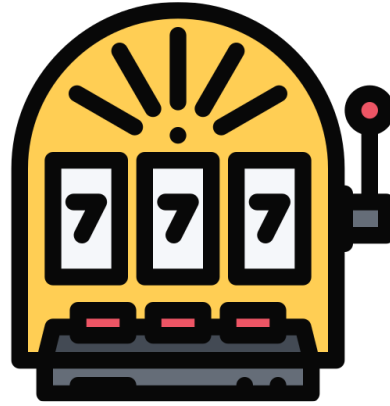
Sutton: Chapter 3-4

What you have learned from bandits

- The need to tradeoff exploitation and exploration, e.g., by an ε -greedy policy
- The difference between a sample, an estimate, and a true expected value

$$R_t, \quad Q_t(a), \quad q_*(a)$$

- The difference between the greedy action and the optimal action
- A learning rule. How learning can be seen as computing an average
- The role of the step size α (how it can be too big, or too small, or “1/n”)

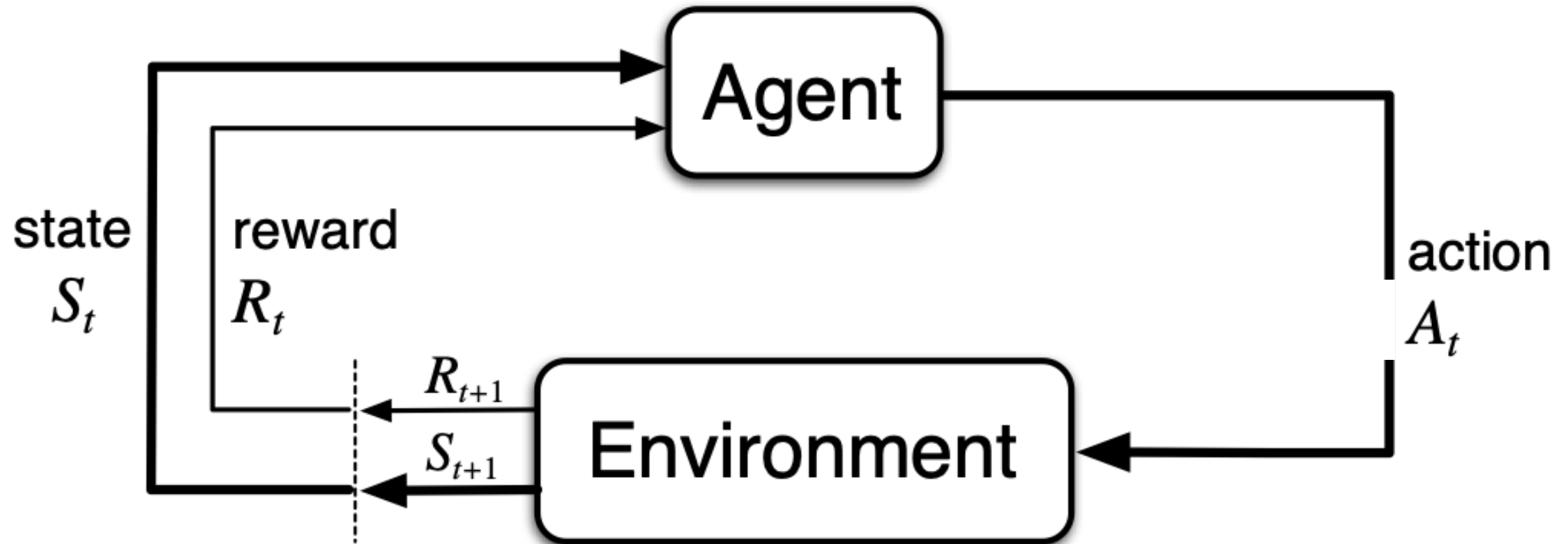


From bandits to MDPs



Non-
deterministic
search

The Agent-Environment Interface



The Agent-Environment Interface

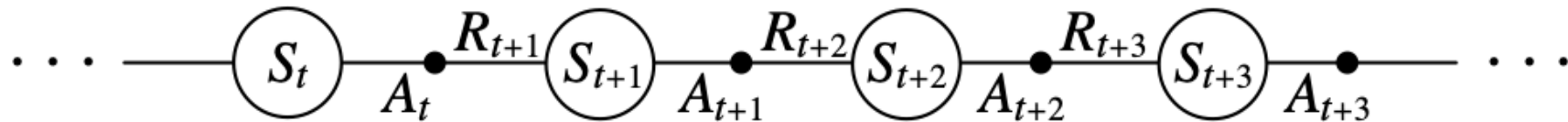
Agent and environment interact at discrete time steps: $t = 0, 1, 2, 3, \dots$

Agent observes state at step t : $S_t \in \mathcal{S}$

performs action at step t : $A_t \in \mathcal{A}(S_t)$

gets the resulting reward R_{t+1} : $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$

and resulting next state: $S_{t+1} \in \mathcal{S}^+$



Example: Grid World

A maze-like problem:

- The agent lives in a grid
- Walls block the agent's path

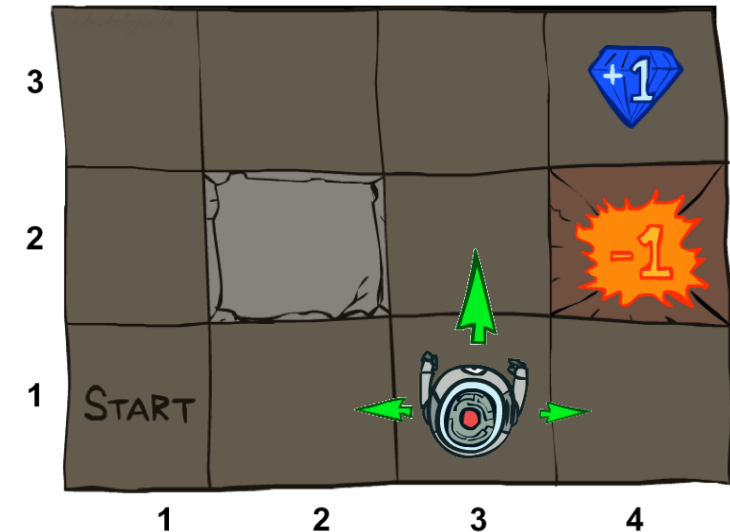
The agent can perform movement actions:

- North, East, South, West
- If there is a wall in the direction the agent would have been taken, the agent stays put

The agent receives rewards each time step

- Small “living” reward each step (can be negative)
- Big rewards come at the end (good or bad)

Goal: maximize sum of rewards



Example: Noisy Grid World

A maze-like problem

- The agent lives in a grid
- Walls block the agent's path

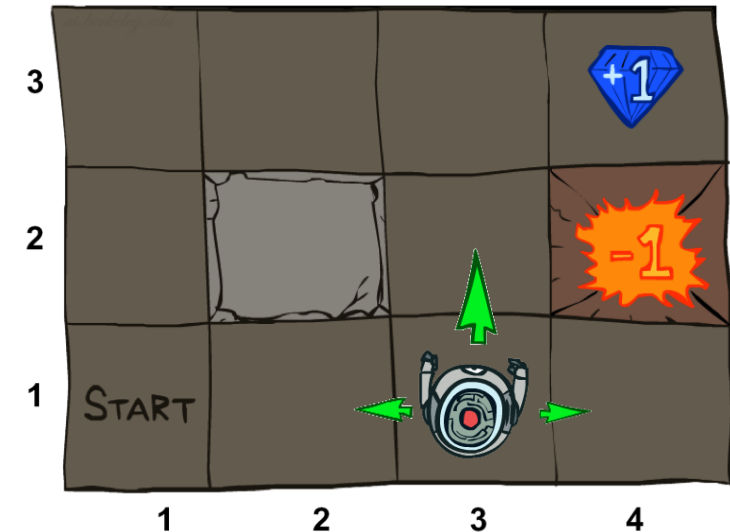
Noisy movement: actions do not always go as planned

- 80% of the time, the action North takes the agent North (if there is no wall there)
- 10% of the time, North takes the agent West; 10% East
- If there is a wall in the direction the agent would have been taken, the agent stays put

The agent receives rewards each time step

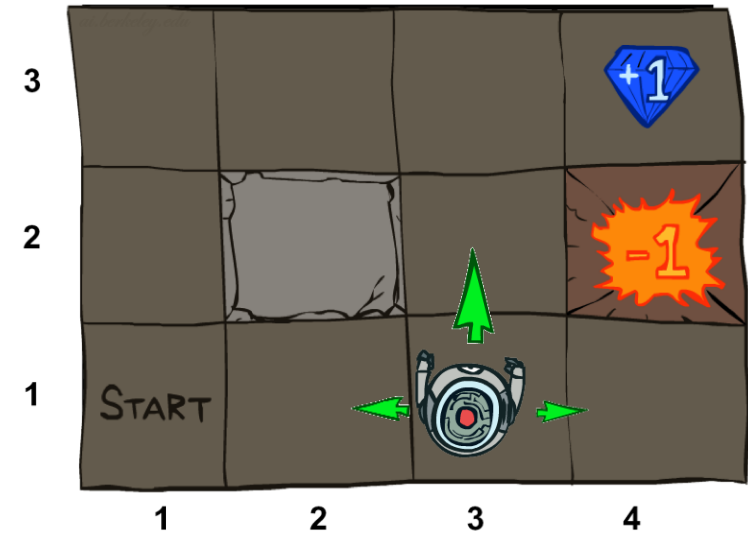
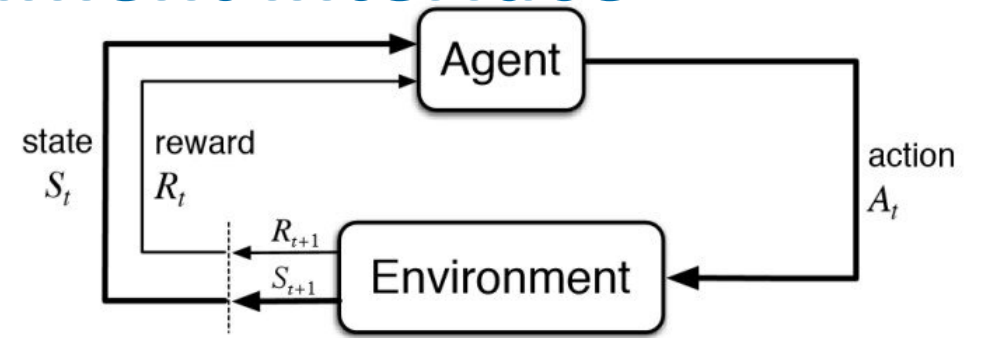
- Small “living” reward each step (can be negative)
- Big rewards come at the end (good or bad)

Goal: maximize sum of rewards



Grid world in the agent-environment interface

- A set of states $S \in \mathcal{S}$
- ... with a **start state** S_0
- A set of actions $A \in \mathcal{A}$
- A reward we get because of taking an action in a state $R \in \mathcal{R}$
- This gives us a trajectory $S_0 A_0 R_1 S_1 A_1 R_2 S_2 A_2 \dots$
- A probability function $p(s', r \mid s, a)$
 - Probability that a from s leads to s' , i.e., $p(s', r \mid s, a)$
 - Also called the model or the dynamics



Markov Decision Process

(MDP)

Markov property

$$P(\text{future} \mid \text{present, past}) = P(\text{future} \mid \text{present, ~~past~~})$$

Markov property 

Markov Decision Process

Markov Decision Process (MDP) is a 5-tuple: $\langle \mathcal{S}, \mathcal{A}, \mathcal{R}, p, \gamma \rangle$

- $\mathcal{S}$: the set of all possible states (*state space*)
- $\mathcal{A}$: the set of all possible actions (*action space*)
- $\mathcal{R}$: the set of all possible rewards
- p : transition probabilities (*dynamics*)
- γ : discount factor

In a **finite** Markov Decision Process, the sets of states, actions, and rewards ($\mathcal{S}$, $\mathcal{A}$, and $\mathcal{R}$) all have a finite number of elements

Markov Decision Process

In a finite MDP, the random variables R_t and S_t have well defined discrete probability distributions dependent only on the preceding state and action:

$$p(s', r | s, a) \doteq \Pr\{S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a\}$$

For all $s', s \in \mathcal{S}$, and $a \in \mathcal{A}(s)$

The function p defines the dynamics of the MDP.

The function p specifies a probability distribution for each choice of s and a :

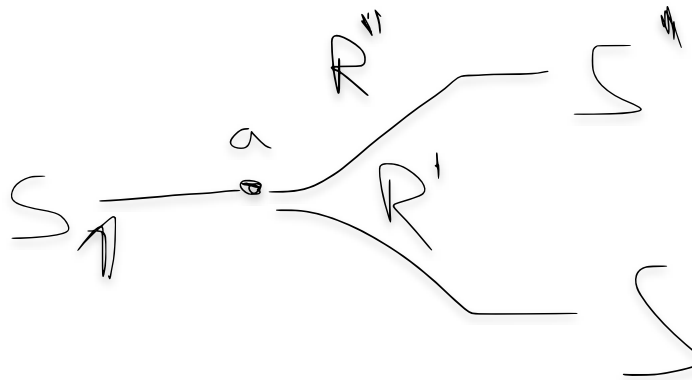
$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r | s, a) = 1, \text{ for all } s \in \mathcal{S}, a \in \mathcal{A}(s)$$

Markov Decision Process

From the dynamics function $p(s', r | s, a)$ we can compute everything we want to know about the environment.

State-transition probabilities:

$$p(s' | s, a) \doteq \Pr\{S_t = s' \mid S_{t-1} = s, A_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r | s, a)$$



s, a		s, r

Markov Decision Process

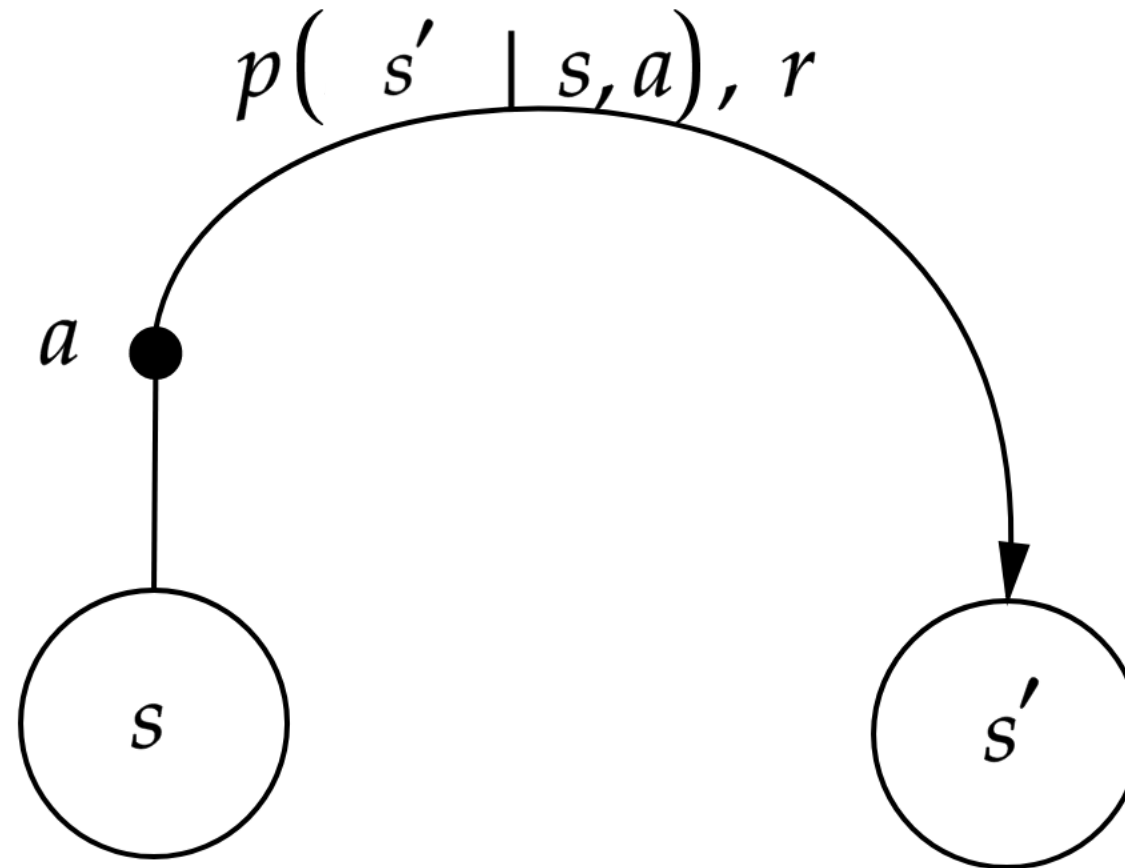
From the dynamics function $p(s', r | s, a)$ we can compute everything we want to know about the environment.

Expected rewards for state-action pairs:

$$r(s, a) \doteq \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r \mid s, a)$$

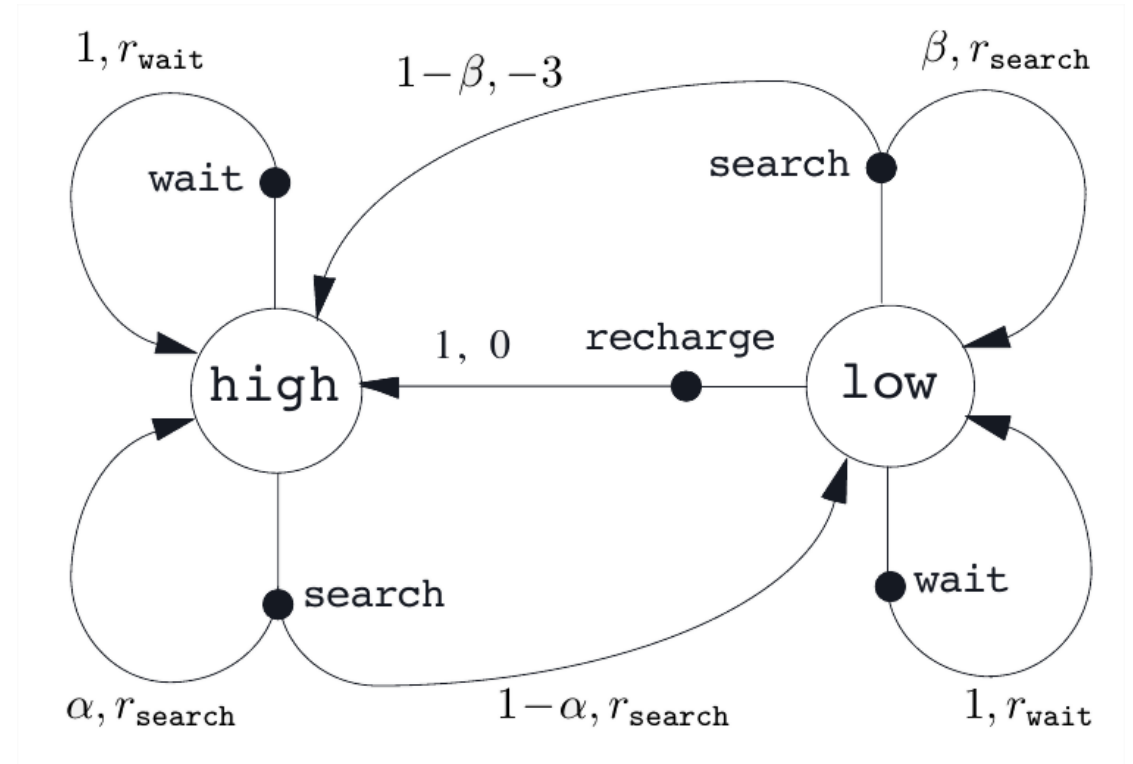
Examples

Transition graphs

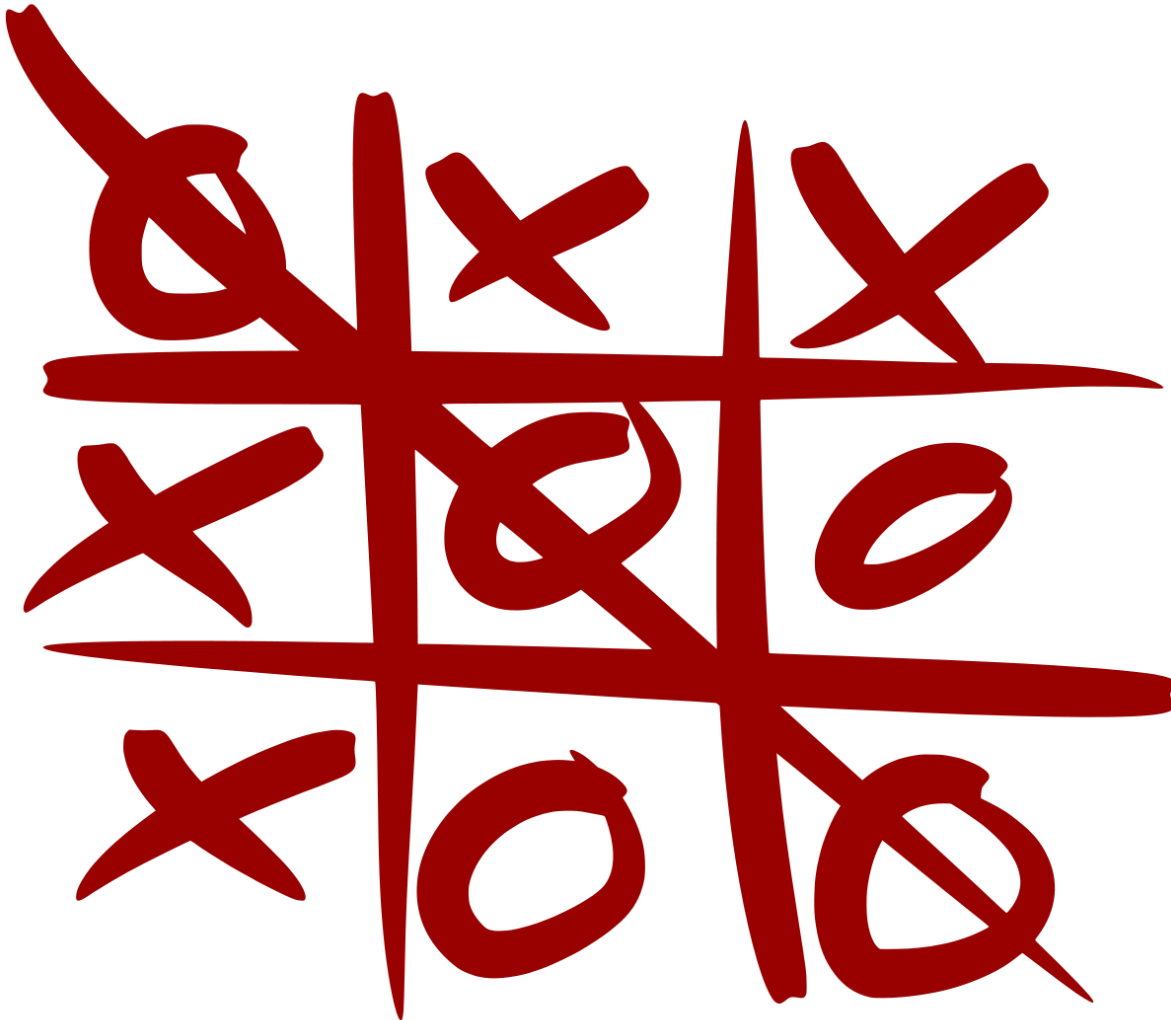


Recycling Robot

s	a	s'	$p(s' s, a)$	$r(s, a, s')$
high	search	high	α	r_{search}
high	search	low	$1 - \alpha$	r_{search}
low	search	high	$1 - \beta$	-3
low	search	low	β	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	-
low	wait	high	0	-
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	-



Tic tac toe



State space

all possible configurations of X and O
+ empty configuration

Action

Putting an X or O at position Y

Reward

- 1 if you win
- -1 if you loose
- 0 if nothing happens or it's a draw

Chess



State space

all possible board configurations
(approximately 10^{46})

Action

Putting piece X at position Y

Reward

- 1 if you win
- -1 if you loose
- 0 if nothing happens or it's a draw



Goals and rewards

What do we want to maximize?

~~Short-term reward: R_t ?~~

The total amount of reward it receives over time (**expected return**)



Reward hypothesis

“

All what we mean by goals and purposes can be well thought of as the maximization of the expected value of the cumulative sum of a received scalar signal (the reward)

”

How to define the reward?

The reward signal should communicate **what** you want the agent to achieve, not **how** the agent should achieve it.

Goal is to maximize the (expected) return



Return

In the simplest case, the **return** G_t is the sum of the rewards:

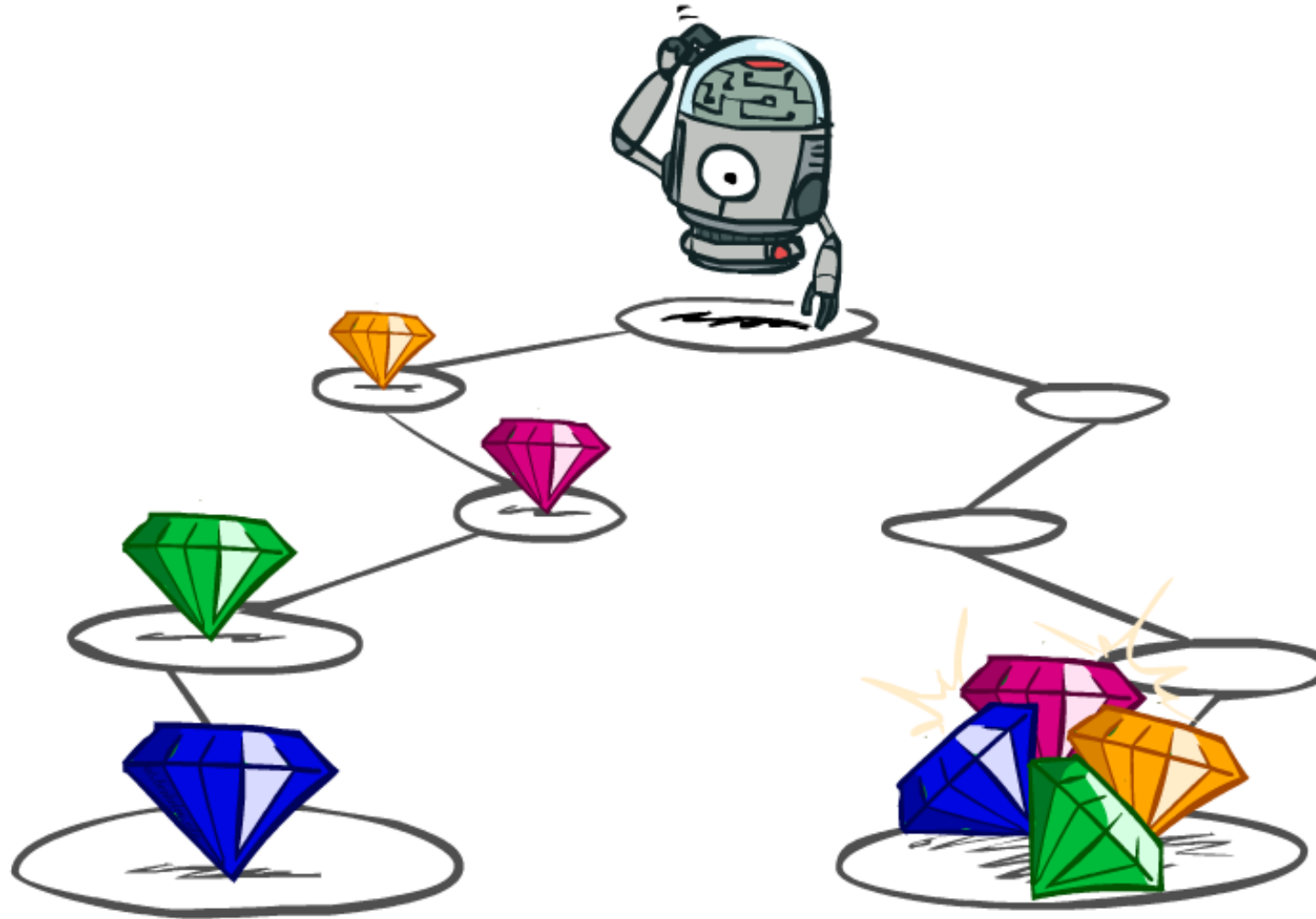
$$G_t \doteq R_{t+1} + R_{t+2} + R_{t+3} + \cdots + R_T$$

where T is a final time step.

We use this approach in **episodic tasks**, where the agent-environment interaction breaks naturally in subsequences (“episodes”), e.g., plays of a game, trips through a maze.

Each episode ends in a special state called the **terminal state**.

Continuing tasks - But what if T is infinite?



Different reward sequences

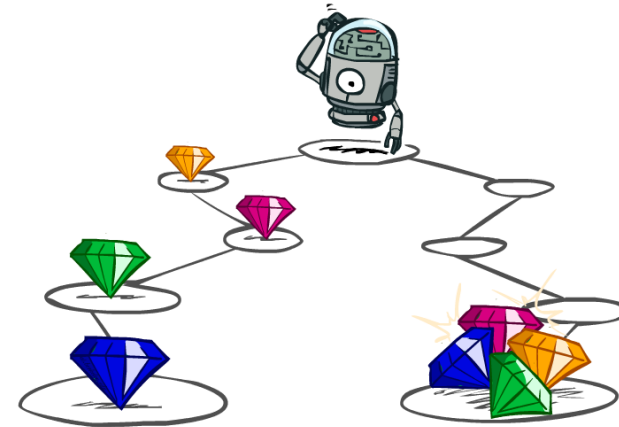
- What preferences should an agent have over reward sequences?

- More or less?

$[1, 2, 2]$ or $[2, 3, 4]$

- Now or later?

$[0, 0, 1]$ or $[1, 0, 0]$



Discounting

- It's reasonable to maximize the sum of rewards
- It's also reasonable to prefer rewards now to rewards later
- One solution: values of rewards decay exponentially



1

Worth Now



γ

Worth Next Step



γ^2

Worth In Two Steps

Discounted Return

Discounted return:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

The **discount rate** γ determines the present value of future rewards: a reward k time steps in the future is only worth γ^{k-1} times what it would be worth if it were received immediately.

Note: $0 \leq \gamma \leq 1$

Example

Assume a discount rate $\gamma = 0.5$.

Given the rewards 1, 2, 3:

$$G_t = 1 + 0.5 \cdot 2 + 0.25 \cdot 3 = 2.75$$

Given the rewards 3, 2, 1:

$$G_t = 3 + 0.5 \cdot 2 + 0.25 \cdot 1 = 4.25$$

Why discounting

- Sooner rewards probably do have higher return than later rewards.
- Helps our algorithms converge.

Discounted Returns

Returns at successive time steps are related to each other:

$$\begin{aligned} G_t &\doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots \\ &= R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \cdots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$

Infinite (continuing) tasks?

Problem: what if our tasks lasts forever? Do we get infinite returns?

Solutions:

- Finite horizon: terminate tasks after a fixed T time steps.
- Discounting: use $0 \leq \gamma \leq 1$

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Example: assume the reward is a constant +1 at every step:

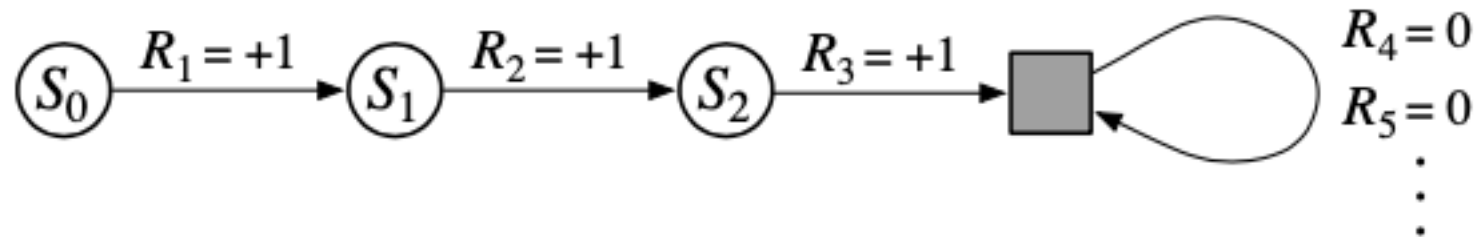
$$G_t = \sum_{k=0}^{\infty} \gamma^k = \frac{1}{1 - \gamma}$$

Unified notation for episodic and continuing tasks

For continuing tasks, we have defined the discounted return as a sum over an infinite number of terms:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

We can use the same definition for episodic tasks if we introduce a special **absorbing state**:

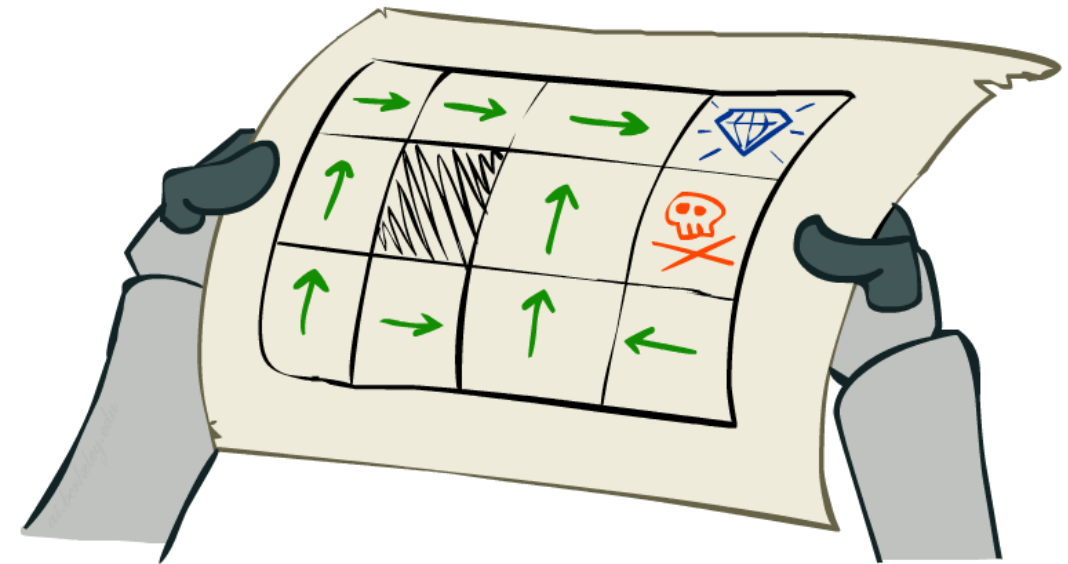


Policies and value functions

Policies

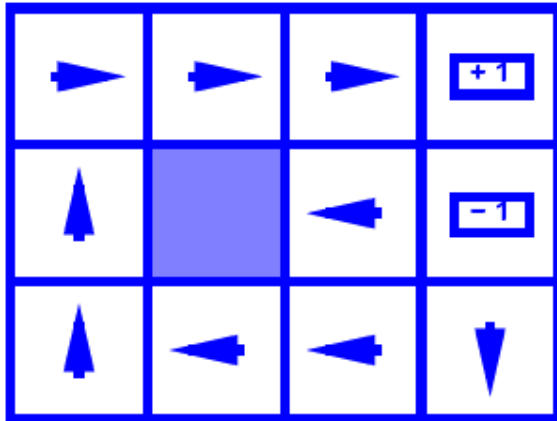
- For MDPs, we want an optimal **policy** π^*
 - A policy π is a mapping from states to probabilities of selecting each possible action.
 - Probability can be 1
 - An optimal policy is one that maximizes the expected return if followed
- 
- A cartoon illustration of a hand holding a rolled-up treasure map. The map is unrolled to show a path of green arrows leading from left to right. The final destination is marked with a blue diamond icon. The map is yellowed and has some scribbles, suggesting it's an old or hidden map.

$\pi(a \mid s)$ = the probability of taking action a in state s , according to policy π

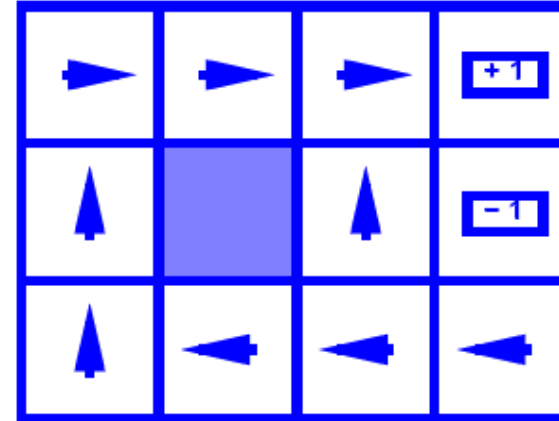


Optimal policy when $R(s, a, s') = -0.03$
for all non-terminals s

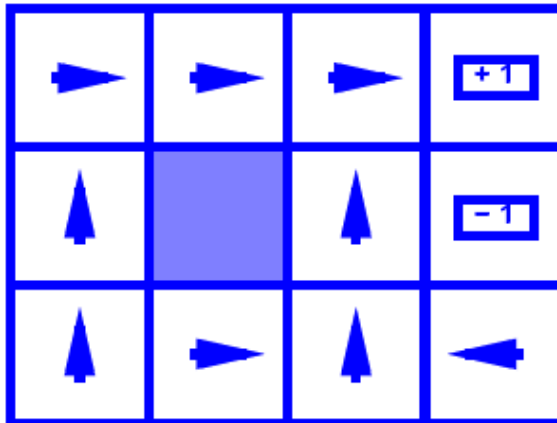
Optimal Policies π^*



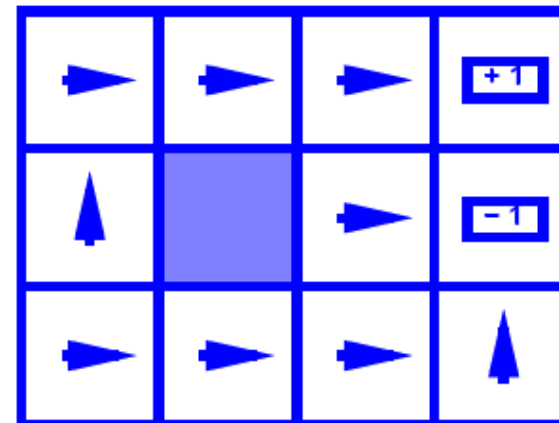
$$R(s) = -0.01$$



$$R(s) = -0.03$$



$$R(s) = -0.4$$



$$R(s) = -2.0$$

Value functions

State value functions

State-value functions are functions of state that estimate how good it is for the agent to be in a given state.

State-action value functions

State-action value functions are functions of state-action pairs that estimate how good it for the agent to perform a given action in a state.

“How good” here is defined in terms of expected return.

Value functions

The rewards the agent can expect to receive in the future depend on what actions it will take.

➔ Value functions are defined with respect to policies.

State-value functions

State-value function for a policy π

We define the value of a state s under a policy π denoted $v_\pi(s)$, as the expected return when starting in s and following π thereafter.

$$v_\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right], \text{ for all } s \in \mathcal{S},$$

$\mathbb{E}_\pi[.]$ denotes the expected value of a random variable, given that the agent follows policy π , and t is any time step. Note that the value of the terminal state, if any, is always zero.

Action-value functions

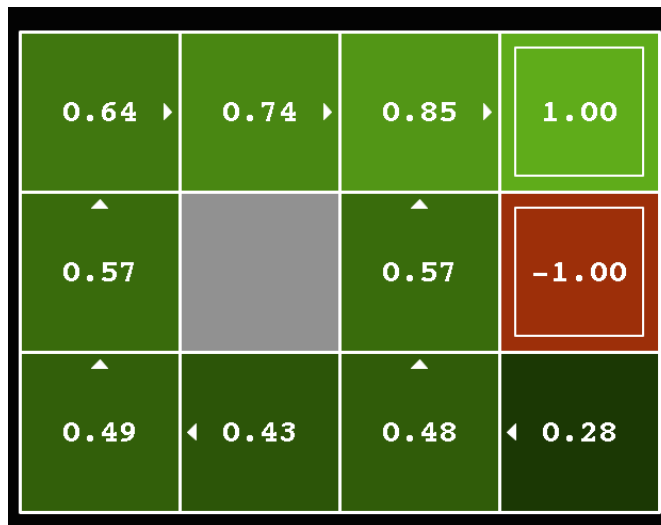
Action-value function for a policy π

We define the value of taking action $\mathbf{a}$ in state $\mathbf{s}$ under a policy π , denoted $q_\pi(\mathbf{s}, \mathbf{a})$, as the expected return starting from $\mathbf{s}$, taking the action $\mathbf{a}$, and thereafter following policy π :

$$q_\pi(s, a) \doteq \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right]$$

Value functions

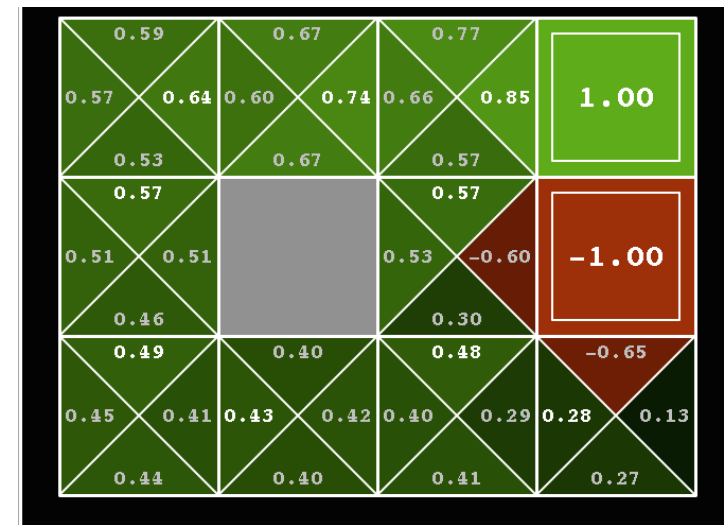
Suppose we have a policy π



How good is it to be
in this state?

state value function for our policy

$$v_{\pi}(s) \doteq \mathbb{E}_{\pi}[G_t \mid S_t = s]$$



How good is it to take an
action in this state?

action value function for our policy

$$q_{\pi}(s, a) \doteq \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

Golf

State is ball location

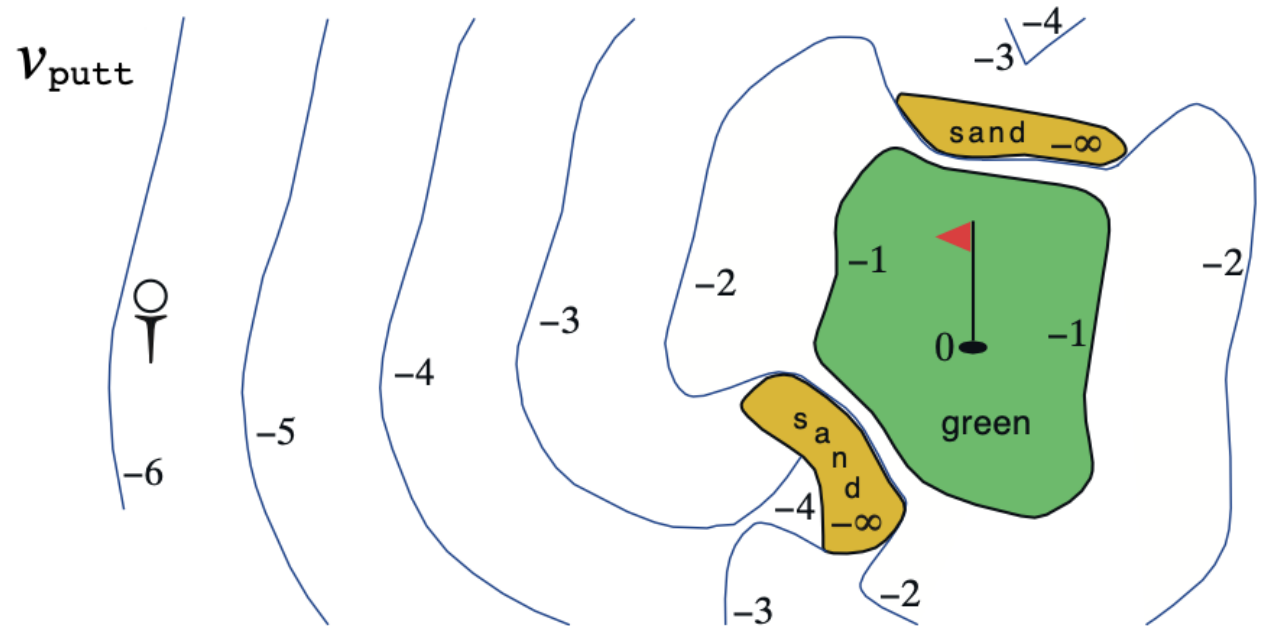
Reward of -1 for each stroke until
the ball is in the hole

Value of a state?

Actions:

- **putt (use putter)**
- **driver (use driver)**

putt succeeds anywhere on the
green



Optimal Policies and Optimal Value Functions

Value functions define a **partial ordering** over policies. A policy π is defined to be better than or equal to a policy π' if its expected return is greater than or equal to that of π' for all states: $\pi \geq \pi'$ if and only if $v_\pi(s) \geq v_{\pi'}(s)$ for all $s \in S$.

Optimal policy: there is always at least one policy that is better than or equal to all other policies. Although there may be more than one, we denote all optimal policies by π_* .

Optimal policies share the same state-value function, called the **optimal state-value** function denoted v_* , and defined as

$$v_*(s) \doteq \max_{\pi} v_{\pi}(s)$$

for all $s \in S$

Optimal Policies and Optimal Value Functions

Optimal policies also share the same action-value function, called the **optimal action-value function** denoted q_* , and defined as:

$$q_*(s, a) \doteq \max_{\pi} q_{\pi}(s, a)$$

for all $s \in \mathcal{S}$ and $a \in \mathcal{A}$.

For the state- action pair (s, a) , this function gives the expected return for taking action a in state s and thereafter following an optimal policy.

Golf

State is ball location

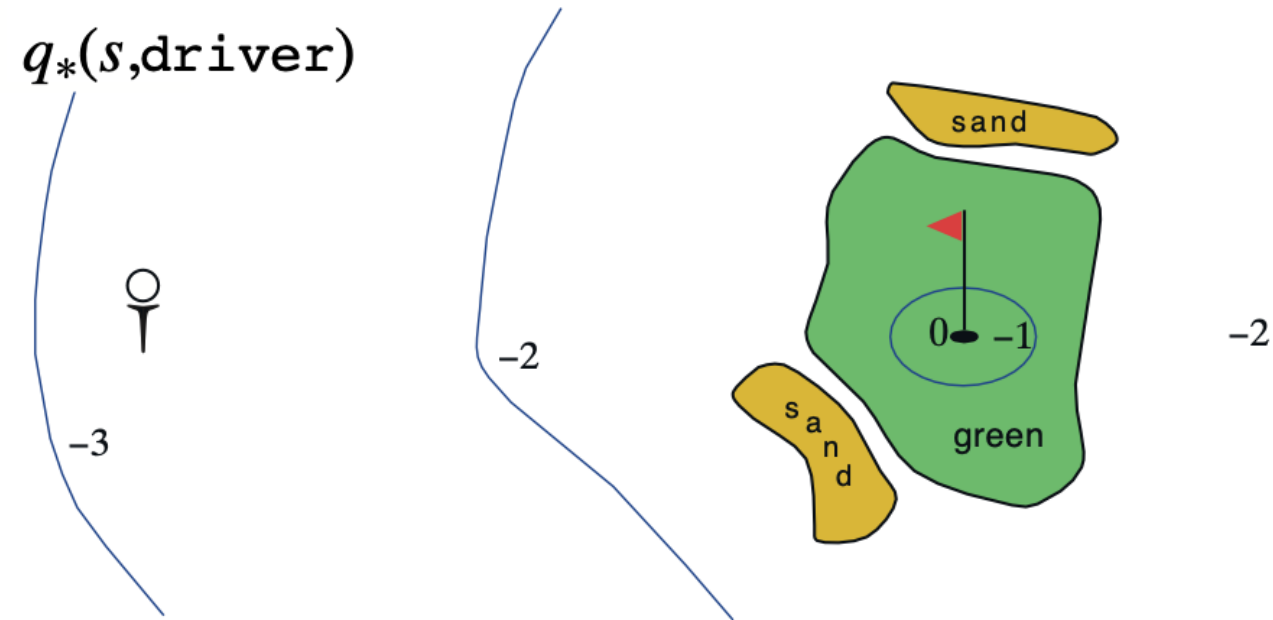
Reward of -1 for each stroke until
the ball is in the hole

Value of a state?

Actions:

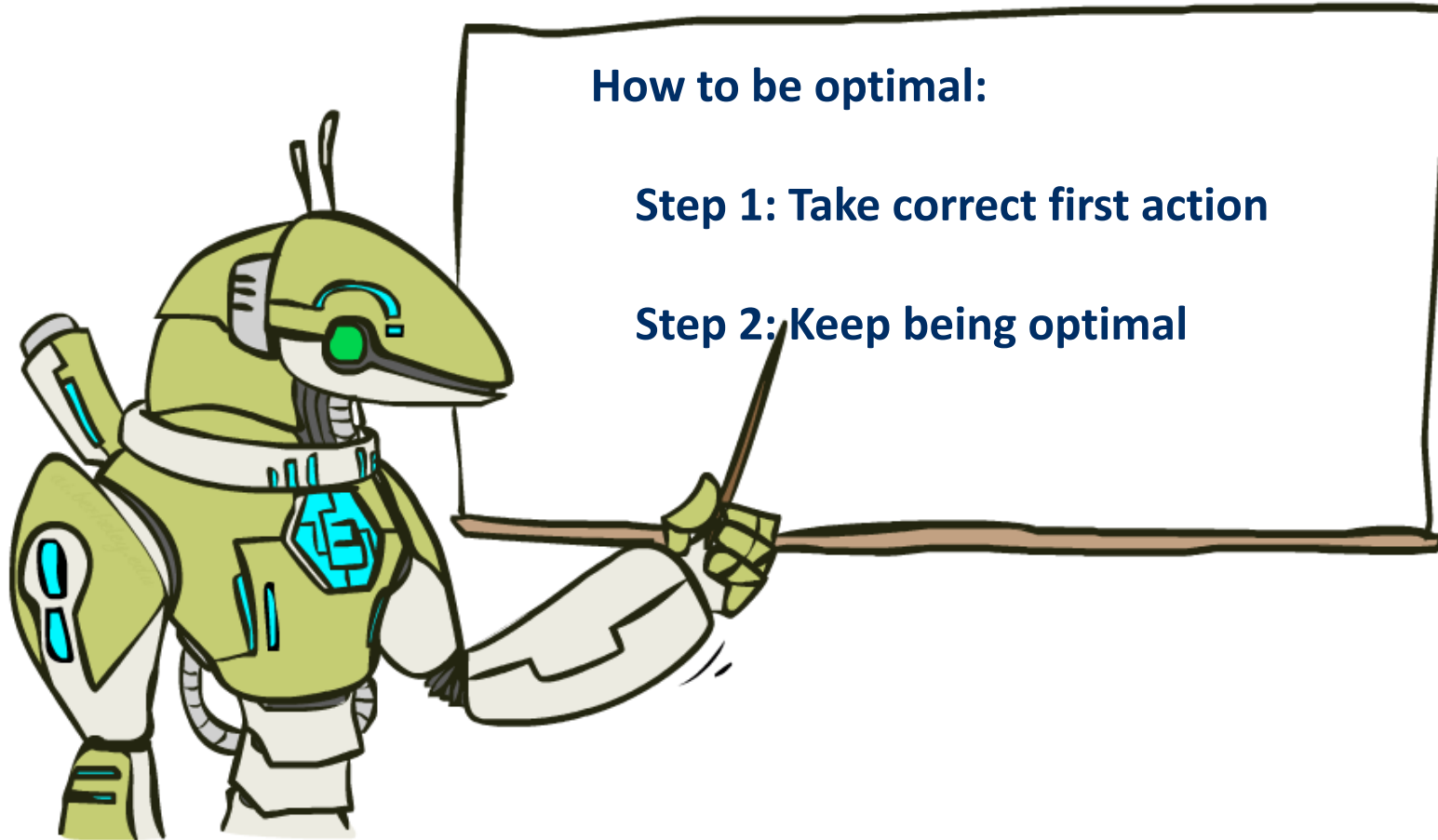
- **putt (use putter)**
- **driver (use driver)**

putt succeeds anywhere on the
green



Finding an optimal policy

The Bellman Equations



Bellman Equation for a Policy π

The basic idea:

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \\ &= R_{t+1} + \gamma \left(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \dots \right) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$

So:

$$\begin{aligned} v_\pi(s) &= E_\pi \left\{ G_t \mid S_t = s \right\} \\ &= E_\pi \left\{ R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s \right\} \end{aligned}$$

Bellman Equation for v_π

A **fundamental property** of value functions used throughout reinforcement learning and dynamic programming is that they satisfy **recursive relationships**.

$$\begin{aligned} v_\pi(s) &\doteq \mathbb{E}_\pi[G_t \mid S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r | s, a) \left[r + \gamma \mathbb{E}_\pi[G_{t+1} | S_{t+1} = s'] \right] \\ &= \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) \left[r + \gamma v_\pi(s') \right], \quad \text{for all } s \in \mathcal{S}, \end{aligned}$$

The **Bellman Equation** for $v_\pi(s)$ expresses a relationship between the value of a state and the values of its successor states.

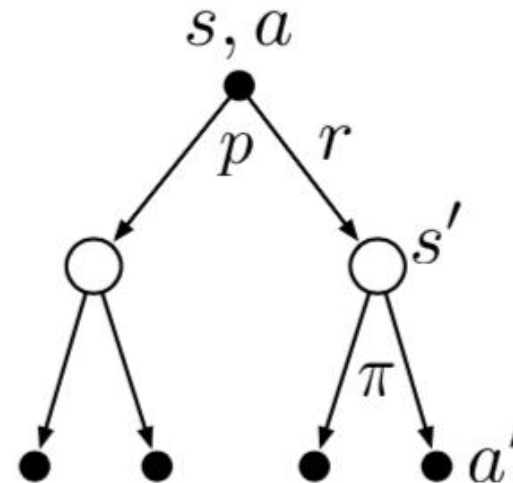
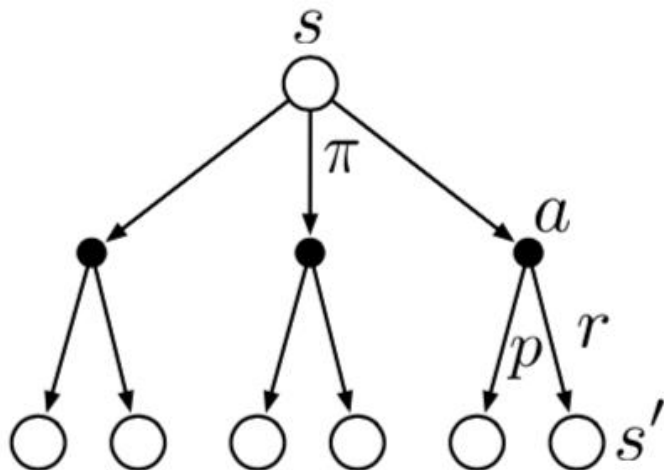
More on the Bellman equations

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) \left[r + \gamma v_{\pi}(s') \right]$$

**This is a set of equations (in fact, linear), one for each state.
The value function for π is its unique solution.**

Backup Diagram for v_π and q_π

Backup diagrams show relationships that form the basis of the update or *backup* operations at the heart of RL methods. These operations transfer value information *back* to a state (or state-action pair) from its successor states (or state-action pairs).

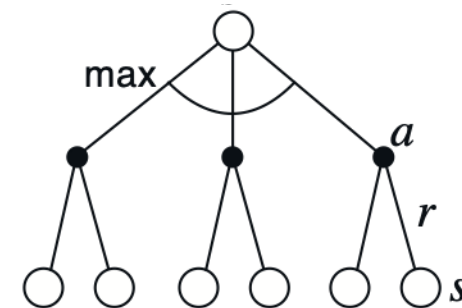


How to find the optimal v_* ?

The value of a state under an optimal policy must equal the expected return for the best action from that state:

$$\begin{aligned} v_*(s) &= \max_{a \in \mathcal{A}(s)} q_{\pi_*}(s, a) \\ &= \max_a \mathbb{E}_{\pi_*}[G_t \mid S_t = s, A_t = a] \\ &= \max_a \mathbb{E}_{\pi_*}[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a] \\ &= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_*(s')]. \end{aligned}$$

The relevant diagram:

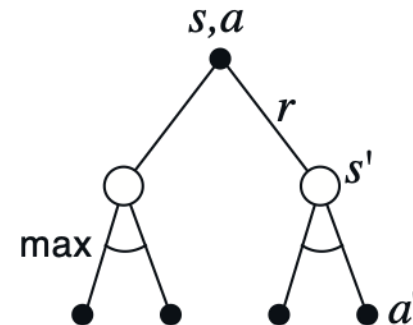


v_* is the unique solution of this system of nonlinear equations.

Bellman Optimality Equation for q_*

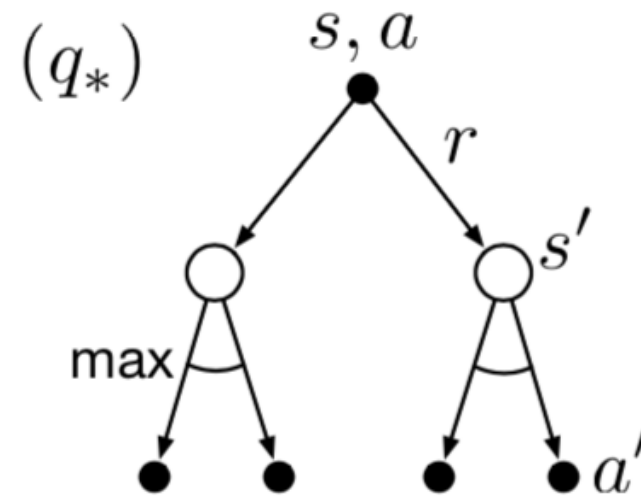
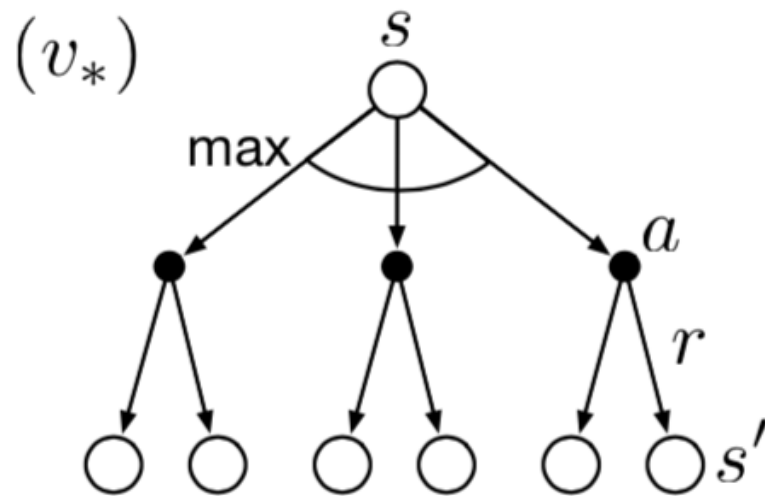
$$\begin{aligned} q_*(s, a) &= \mathbb{E} \left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a \right] \\ &= \sum_{s', r} p(s', r | s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right]. \end{aligned}$$

The relevant backup diagram:



Backup Diagram for v_* and q_*

These are the same as the backup diagrams for v_π and q_π presented earlier except that arcs have been added at the agent's choice points to represent that the *maximum* over that choice is taken rather than the expected value given some policy.



Calculating this set of equations

Easy!

- If you have enough space and time to do the computation.
- If you know the model
- If it's an MDP

But...

- Polynomial in state
- State space is often too big
- We must resort to approximation techniques